

SEOS-2857 os_network / if_OS_Socket update

Page Content

- [Introduction](#)
 - [Current Situation](#)
 - [Target](#)
- [Proposed Solution](#)
 - [CAmKES Architecture](#)
 - [Scope of change](#)
- [Draft Implementation Details](#)
 - [Files](#)
 - [Notification concept](#)
 - [Non-blocking operation](#)
 - [Network Stack CAmKES Component](#)
 - [Others](#)
 - [State diagram](#)
- [Open Points](#)
- [WIP - Notification Scheme Ideas](#)
 - [Idea Notification scheme A](#)
 - [Idea Notification scheme B](#)
 - [TimeServer / Timeouts](#)
 - [New RPC functions](#)
 - [Network Stack](#)
 - [Network Client](#)
- [Alternative solutions](#)
 - [Custom connector](#)
 - [Design targets / considerations](#)
 - [Constraints / Limitations](#)
 - [Design](#)
 - [RPC](#)
 - [Custom connector](#)
 - [Client](#)
 - [Network stack](#)
 - [Operation](#)
- [Simplifying design](#)
- [Development Process](#)
 - [Workflow](#)
 - [Proposed Work Packages](#)
 - [Adapt the API](#)
 - [Replace hardcoded dependency between RPC interface name and library functions](#)
 - [Adapt the if_OS_Socket CAmKES Interface](#)
 - [Implement handle management based on Client ID](#)
 - [Implement the new notification scheme in the os_network_stack library](#)
 - [Implement the NetworkStack_PicoTcp SDK component](#)
 - [Adapt all demo/test systems that use networking to make use of the NwStack SDK component.](#)
 - [Adapt the TLSLib and TLSServer to the new CAmKES Interface of the NwStack](#)
 - [Update the SDK Handbook documentation, Release Notes, and the migration guide](#)
 - [Possible extensions](#)
 - [Extend Client library to support blocking socket calls](#)
 - [Extend API with an additional function to set socket options.](#)
 - [Split current network stack library implementation into generic functions and implementation-specific \(e.g. picoTCP\) parts](#)
 - [Add demo "multi-protocol-echo-server"](#)
- [Proposed new API functions](#)
 - [API Functions](#)
 - [API Use-Cases](#)
 - [TCP Client](#)
 - [Application \(user1\) connects to Server, sends a request, handles response, and closes the socket](#)
 - [Application connects to Server, tries to send Data but loses connection and tries to reconnect](#)
 - [Application opens several sockets to connect to several Servers, exchanges data.](#)
 - [TCP Server](#)
 - [Server listens for incoming client connections, no client connects](#)
 - [Server listens for incoming client connections, one client connects, sends a short message, and immediately disconnects.](#)
 - [Server listens for incoming client connections, several clients connect](#)
 - [Server listens for incoming client connections, several clients connect at the same time](#)
 - [UDP Client](#)
 - [Application \(user1\) connects to Server, sends a request, handles response, and closes the socket](#)
 - [UDP Server](#)
 - [Server listens for incoming data on a bound port, one client sends some data.](#)
 - [Server listens for incoming data on a bound port, several client send data.](#)
- [Epic: Non-blocking / single dataport architecture for if_OS_Socket](#)

Introduction

Current Situation

Our current `os_network` API / `if_OS_socket` implementation uses a concept, in which for each socket the network stack holds, it uses one unique dataport. So if a network stack shall offer up to 16 open sockets in parallel, it needs to offer 16 dataports. As the `os_network` API connects those dataports directly to its socket management array (element 1 = dataport 1, element 2 = dataport 2, ...) and the `os_network` API is unaware of its Clients, each Client component of the network stack needs to connect to all available dataports. This ultimately leads to several Clients sharing all available dataports of the network stack.

Another drawback of the current implementation is, that it is difficult to establish a default network stack CAMkES component, as the number of supported sockets (which should be a simple configuration value), has a direct influence on the architecture (number of dataports) and the Client connection scheme.

Below is the (linked) architecture diagram of the current implementation.



Source page access error: Does the page from the link below exist?

</pages/viewpage.action?pageId=32310040>

Target

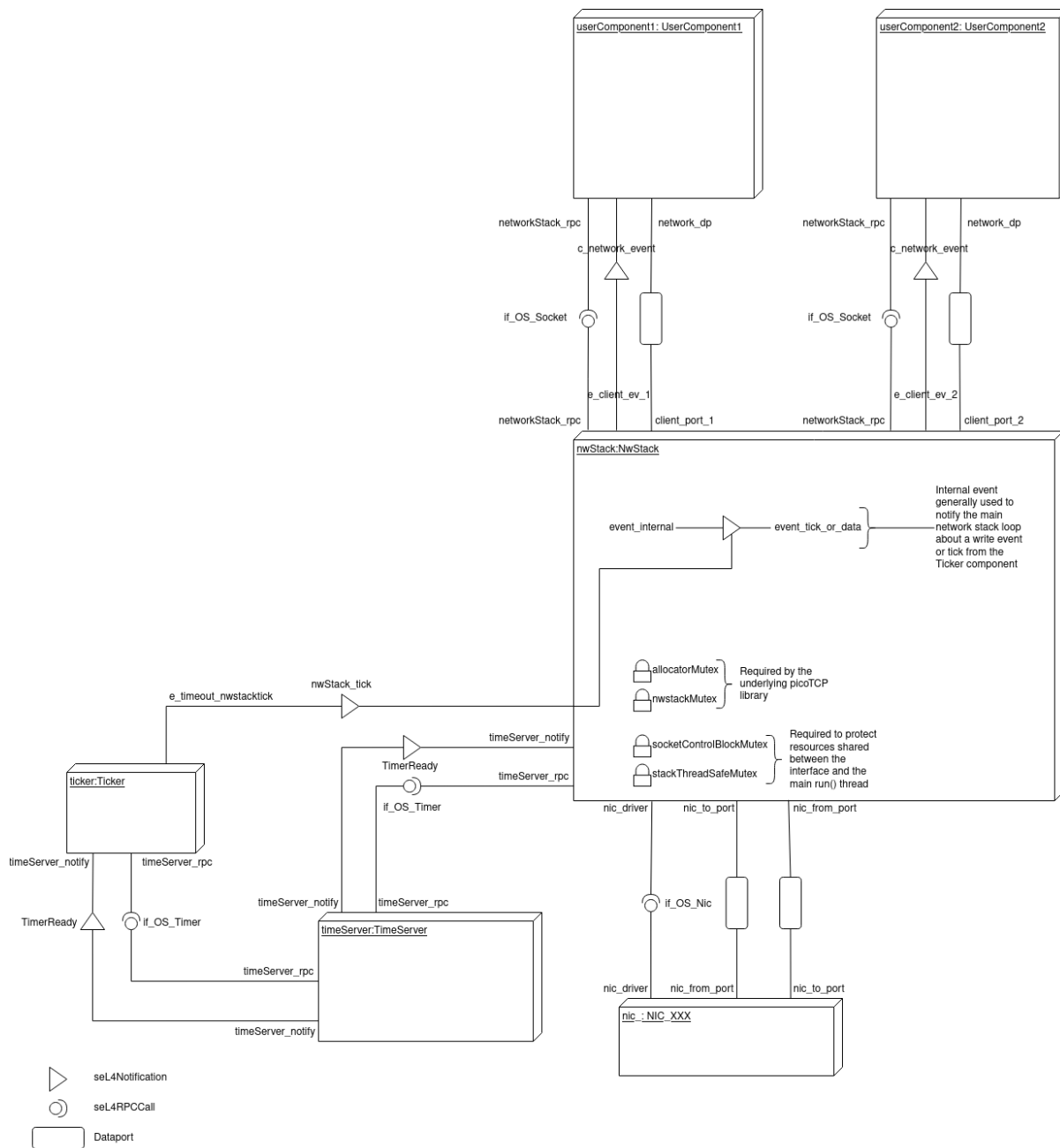
The target of the ticket is, to change the connection scheme between network stack component and Clients, so that:

- each Client uses 1 RPC and 1 dataport, similar to e.g. `StorageServer / if_OS_Storage`
- a Client can manage more than 1 socket over the single dataport
- To avoid the negative impact of Clients on the network stack, the number of sockets per Client should be limited by:
 - a) the total number of sockets allowed by the network stack
 - the number needs to be a define because it will need to be known at compile time for the Network Stack component.
 - should be a define/configuration parameter - possibly a CAMkES configuration parameter, we could set during component definition
 - is the sum of all opened sockets of all Clients
 - Target for the configurable range should be 1..64 (needs to be discussed, what is a meaningful value)
 - b) the number of sockets allowed for the Client
 - should be a CAMkES configuration parameter, similar to `StorageServer_COMPONENT_DEFINE / storageServer_config - offset /size`
 - Target for the configurable range should be 1..64 (needs to be discussed, what is a meaningful value)
- There should be a Client connect macro similar to `StorageServer_INSTANCE_CONNECT_Clients`
- the network stack component / `if_OS_Socket / os_network` should allow up to 8 connected CAMkES Clients
- the network stack shall prohibit one Client from interfering with another Clients sockets
- the network stack, especially the Clients RPC interface, shall implement a non-blocking scheme
 - as RPC calls are inherently blocking, the RPC functions shall be designed, so that they will return in a reasonable short duration
 - RPC functions should not contain wait or sleep procedures
 - Socket operations shall be implemented by RPC calls, so that blocking network operations (accept, read, write, etc.) are not performed within the RPC call
 - in the case of multiple Clients for one network stack, no Client shall be able to block the network stack for longer periods

Proposed Solution

CAMkES Architecture

The diagram below depicts the new architecture, changes in red.



The interface shall be similar to other implementations in the SDK like:

- CertServer
- CryptoServer
- StorageServer
- TimeServer

In those cases, the Component / Server offers 8 "maybe" dataports to connect to a variable number of Clients. By a macro Clients are connected to the server (e.g. `CertServer_INSTANCE_CONNECT_Clients`).

Via macros `xxx_Client_ASSIGN_BADGES` the Clients are identified and dataports accordingly matched (during runtime, e.g. `CID_TO_IDX`).

Scope of change

The complete scope of the change is as follows (might be incomplete)

- modified `os_network` / `if_OS_network`
- new CAmkES Component for Network Stack

- adapted demos
- adapted test systems (test_network_api)
- updated documentation
 - Handbook Page for Network Stack component
 - Network API description
 - demo descriptions / architectures

Draft Implementation Details

Files

path	filename	brief change description
os_core_api/camkes	if_OS_Socket.camkes	none obvious
os_core_api/include	OS_Network.h	▪ clarify the use of ctx attributes ▪ update and review comments ("only TCP supported")
os_core_api/include/network	OS_Ethernet.h	none obvious
os_core_api/include/network	OS_Network_types.h	none obvious
os_core_api/include/network	OS_NetworkStack.h	▪ none obvious ▪ why has it the same name as libs/os_network_stack/include/OS_NetworkStack.h? ▪ The contained struct is relevant for the if_OS_Nic interface. So it would better fit with some NIC name. ▪ It would be better to split the xxx_network_xxx files into ▪ xxx_socket_xxx for things belonging to the if_OS_Socket interface (needs to be used by NetworkStack and its Clients) ▪ xxx_nic_xxx for things belonging to the if_OS_NIC interface (needs to be used by NetworkStack and NIC) ▪ xxx_network_xxx/xxx_nwstack_xxx for things used NetworkStack internally
libs/os_network_stack	network_Client.c	▪ get_data_port() probably not required any more ▪ OS_NetworkStackClient_init() probably not required any more ▪ change dataport assignment in: ▪ OS_NetworkSocket_read() ▪ OS_NetworkSocket_rcvfrom() ▪ OS_NetworkSocket_write() ▪ OS_NetworkSocket_sendto()
libs/os_network_stack	network_stack.camkes	• move to new component • modifications of most macros necessary, similar to CertServer/CryptoServer/StorageServer ◦ add maybe dataports ◦ add Client connect macros ◦ add badge macros ◦ add config macros (max_total_sockets; max_sockets_of_each_Client)
libs/os_network_stack/include	OS_NetworkStack.h	▪ possibly OS_NetworkStack_CamkesConfig_t to remove/modify dataport configs ▪ OS_NetworkStack_SocketResources_t: add attribute for CID (identification of Client that opened the socket) ▪ possibly more
libs/os_network_stack/include	OS_NetworkStackClient.h	▪ remove OS_NetworkStackClient_SocketDataports_t ▪ remove OS_NetworkStackClient_init() ▪ Why are the prototypes for the rpc calls defined in here? Isn't that created during the build process by the camkes code generator?

libs /os_network_stack /network_core/src	network_conf ig.c	- internal signalling via read/write/connection events need to be modified. This was a per socket/dataport scheme. As we are now separating sockets and dataports, we need to rethink this strategy - details to be drafted
libs /os_network_stack /network_core/src	network_conf ig.h	unclear
libs /os_network_stack /network_core/src	network_stack_core.c	- adaption to new handle/socket handling - revision of wait/notify getter functions - inclusion of badge handling - revision of initialisation as needed
libs /os_network_stack /network_core/src	network_stack_core.h	unclear
libs /os_network_stack /network_core/src	network_stack_pico.c	- add badge processing and checking - dataport related changes (mostly hidden behind get_app_port())
libs /os_network_stack /network_core/src	network_stack_pico.h	unclear
libs /os_network_stack /network_core/src	network_stack_pico_nic.c	none obvious
libs /os_network_stack /network_core/src	network_stack_pico_nic.h	none obvious

Notification concept

Currently the network stack uses a notification scheme for connecting the picoTCP implementation with our code.

- the NwStack component included for each dataport/socket the following notifications:
 - emits NwStack_WrEv e_write_##n;
 - consumes NwStack_WrEv c_write_##n;
 - emits NwStack_RdEv e_read_##n;
 - consumes NwStack_RdEv c_read_##n;
 - emits NwStack_ConnEv e_conn_##n;
 - consumes NwStack_ConnEv c_conn_##n;
- each e_xxx and c_xxx are connected to each other
- network_config.c implements wait and notification functions for those events.
 - These are wrappers that are called with a handle. From the handle the software derives which dataport/socket notification is to be used.
 - internal_notify_read()
 - internal_wait_read()
 - internal_notify_write()
 - internal_wait_write()
 - internal_notify_connection()
 - internal_wait_connection()
- Signalling:
 - in the following functions **internal_wait_xxx()** is called to wait for the notification to be received:
 - network_stack_pico_socket_connect()
 - network_stack_pico_socket_accept()
 - network_stack_pico_socket_write()
 - network_stack_pico_socket_read()
 - network_stack_pico_socket_recvfrom()
 - all above notifications are initiated in **handle_pico_socket_event()** by internal_notify_xxx()
 - This setup realizes a blocking operation.
- For the new interface this concept should be modified
 - As a working basis, let's assume:
 - the network stack supports a maximum of 64 sockets
 - 4 Client applications are connected, each one with 5 sockets
 - Client#2 wants to write data on it's 3rd socket
 - Client#4 wants to receive data on it's 4th socket
 - Client#5 wants to open another socket
 - How shall the signalling run?**

Non-blocking operation

The current OS_network implementation is designed to be blocking. It is not only blocking on Client/user side, which is a typical scheme. It is also blocking on network stack side, due to the use of RPC calls. So one network stack Client will block its own and the network stack operation by a blocking socket function like, write or accept.

This scheme cannot scale and can also not be workaround by a user, that requires e.g. 2 network connections, or that wants to implement a server with more than one parallel Client connection.

In the following table, all current RPC functions are listed and thoughts are mentioned on what would need to be changed. This is work in progress.

RPC Function	Description	Called by	Required changes	Comments
socket_create() networkStack_rpc_socket_create()	- creates the internal management structure of the socket in the TRENTOS wrapper part - creates the socket in picoTCP	OS_NetworkSocket_create() OS_NetworkServerSocket_create()	- Implement mechanisms based on badge IDs of the Clients to first verify if the particular Client is still allowed to create a socket or if he has reached his allowed number of sockets (e.g. <code>isClientQuotaUsedUp()</code>, <code>decrementClientQuota()</code>). - Implement functions and extend <code>network_stack_t / OS_NetworkStack_SocketResources_t</code> to support mapping the reserved socket handle to a specific Client ID so the <code>handle_pico_socket_event()</code> can derive the Client ID from the socket handle during its loop to perform certain Client specific checks or possibly Client specific signalling.	
socket_close() networkStack_rpc_socket_close()	- deletes the internal management structure of the socket in the TRENTOS wrapper part (immediately) - sets the socket in picoTCP in the "to be closed" state. Actual closing happens later. (creates some issues with our management structures, that are not available at that time)	OS_NetworkSocket_close() OS_NetworkServerSocket_close()	- change to a socket close request - delete the management structure only after picoTCP closed the socket (could possibly be done checking the socket state in <code>handle_pico_socket_event</code> and then call <code>free_handle()</code> here once the required state has been reached). We would then not also have to free the socket but also increment the Clients socket budget again (implement function <code>incrementClientQuota()</code>). - the Client does not need to be informed about the final closing - Issues could arise from the maximum number of sockets - as long as the socket is not finally closed, it will use one of the available socket elements - It would not count that against the Client's socket budget - Issues can come up in low socket number situations, e.g.: - max number of sockets = 1 - Clients max number sockets = 1 => if the Client wants to reopen a socket right away, it cannot do it => requires documentation	A socket in a <code>time_wait</code> state can take 240 seconds by default until it can be used again.
socket_connect() networkStack_rpc_socket_connect()	- Connect the socket to a server - requires cooperation from an external entity, can therefore block for longer periods - uses <code>internal_wait_connection()</code>, to wait for connection to be established	OS_NetworkSocket_create()	- change to socket connect request - needs to have a new scheme, how the successful/unsuccessful connection is being reported to the Client later on	If we make the connect call in the Client lib blocking and on unblocking we call connect again, we can change the function there to return the error code.
socket_accept() networkStack_rpc_socket_accept()	- waits for connection notification on the handle with <code>internal_wait_connection(handle)</code>; - after reception of the notification, it looks up the attribute "accepted_handle" of the handle management structure - this attribute refers to the last accepted Client connection handle, this is also returned - the actual acceptance of Client connections are done via <code>pico_socket_accept()</code> in <code>handle_incoming_connection()</code>; this in turn is called in our well known callback function: <code>handle_pico_socket_event</code> - Via the parameter "backlog" one can define, how many pending (not accepted) Client connections shall be managed by picoTCP. If this is exceeded, incoming requests are terminated right away by picoTCP.	OS_NetworkServerSocket_accept()	- will block "accepted_handle" should be changed to "parent_handle". So instead of storing the relationship between server socket and accepted sockets with the server handle, move it to the Client handle. Alternatively one would need to extend <code>accepted_handle</code> to an array, as more than 1 accepted socket is possible per listening socket. - The function should be renamed, as it is not performing the accept. <code>pico_socket_accept()</code> is called automatically in the main processing loop of picoTCP. - The main processing loop logic in <code>handle_incoming_connection()</code> or maybe only the function <code>set_accepted_handle()</code> needs to be reworked so it can queue up several accepted connections that can then be dequeued and picked up the Client. As stated above, currently this function only allows for one accepted socket per listening socket. The max amount would then need to match with the backlog settings. - A new scheme is necessary to inform the Client of incoming Client connections. We could either do a polling scheme or notifications. - Currently, there is no restriction on the allowed Client connections. This should be further implemented. E.g. we could have a configuration parameter for this in <code>OS_NetworkServerSocket_create()</code>. In <code>handle_incoming_connection()</code> we could check for this before we call accept. If we don't accept, the request will be pending. If more than "backlog" requests are pending, picoTCP will discard any more requests. - parameter "port" is not used, seems without purpose	The relationship itself shouldn't be stored. It's just used to return the handle to the Client.
socket_listen() networkStack_rpc_socket_listen()	sets a socket into listening mode	OS_NetworkServerSocket_create()	- this should not block - Does not have a separate API Function, as e.g. <code>socket_bind()</code> <code>network_stack_pico_socket_listen()</code> has a hard coded check, that only sockets with <code>handle == 0</code> can be set to listening mode. This needs to be removed.	Similar as mentioned below and in some comments above the question would be if we still want to bundle these calls behind <code>OS_NetworkServerSocket_create()</code> or if we rework the API to drop all <code>ServerSocket_XXX</code> functions and directly expose the bundled functions.
socket_bind() networkStack_rpc_socket_bind()	binds a socket to a port	OS_NetworkSocket_bind() OS_NetworkServerSocket_create()	- this should not block - no required changes obvious - unclear, whether we need to have that function in the rpc call - instead we could also create <code>socket_create_server()</code> function that is performing the operations of <code>OS_NetworkServerSocket_create()</code> on Network Stack side.	The question is if we want to hide bind and listen behind our functions or not.

socket_write() networkStack_rpc_socket_write()	- waits for write notification on the handle with internal_wait_write(handle); - writes data to the socket	OS_NetworkSocket_write()	- can block on the internal wait_write() - pico_socket_write() should not block - instead of waiting, it should check sockevent for PICO_SOCKET_EV_WR, and return with error if not set - get_app_port() needs to be adjusted - If we do not use the sel4RPCDataportSignal connector currently offered by CAMKES (which already includes functionality for the server to retrieve the Client buf by its ID), we should implement a similar pattern here as we use in the StorageServer get_Client_port(), which also makes use of the badge of the Client to resolve the respective shared memory port for this Client. - Why does it do "sock->event = 0;"?	
socket_read() networkStack_rpc_socket_read()	- waits for read notification on the handle with internal_wait_read(handle); - reads data from the socket - has a while loop to read a certain amount of data	OS_NetworkSocket_read()	- can block, when no data is present - remove while loop on read - only read once, return result - should check for available data (PICO_SOCKET_EV_RD) and return if no data is present - can be extended by signaling of data reception - get_app_port() needs to be adjusted	
socket_sendto() networkStack_rpc_socket_sendto()	writes data to the socket	OS_NetworkSocket_sendto()	- Should not wait for internal_wait_write() / PICO_SOCKET_EV_WR, see PicoTCP docu https://os.mbed.com/media/uploads/daniele/user_doc.pdf page 22 - get_app_port() needs to be adjusted	
socket_recvfrom() networkStack_rpc_socket_recvfrom()	- waits for read notification on the handle with internal_wait_read(handle); - reads data from the socket - has a while loop to read a certain amount of data	OS_NetworkSocket_recvfrom()	- can block, when no data is present - remove while loop on read - only read once, return result - should check for available data (PICO_SOCKET_EV_RD) and return if no data is present - can be extended by signaling of data reception - get_app_port() needs to be adjusted	

Network Stack CAMKES Component

- Needs to be created
- Naming options (not Complete):
 - NetworkStack
 - NwStack
 - IpStack
 - TcpUdpStack
 - NetworkManager
 - ...
- network_stack.camkes shall be removed from libs/os_network
 - instead moved to CAMKES component
- First iteration shall offer 8 "maybe" dataports for connection of up to 8 Clients
- To be clarified: badges starting from 101? Or should we use other numbers?
- Functions /Macros:
 - #define NwStack_COMPONENT_DEFINE
 - #define NwStack_INSTANCE_CONNECT
 - #define NwStack_INSTANCE_CONFIGURE
 - max number sockets - is this a define in C-Code we can use to define the size of the array (OS_NetworkStack_CamkesConfig_t internal.number_of_sockets)
 - #define NwStack_INSTANCE_CONNECT_ClientS
 - One issue we need to solve is, that on the other examples this is being implemented by giving here RPC and dataport of the Clients as attributes.
 - Our current os_network API hides rpc and dataport from the user, e.g.
 - API: OS_NetworkSocket_write() => get dataport by get_data_port(handle)
 - calls network_stack_rpc_socket_write(), which is derived from the default rpc name "network_stack_rpc" used on Client and NwStack side.
 - we could use the scheme implemented in if_CertServer.h / IF_CERTSERVER_ASSIGN, resp. CertServer.c / CertServer_addCertToChain()
 - #define NwStack_INSTANCE_CONFIGURE_ClientS
 - max number of sockets for each Client
 - #define NwStack_Client_ASSIGN_BADGES

Others

- Extend Macros FOR_EACH_1P and FOR_EACH_2P to up to 16 arguments/tuppels (future preparation)

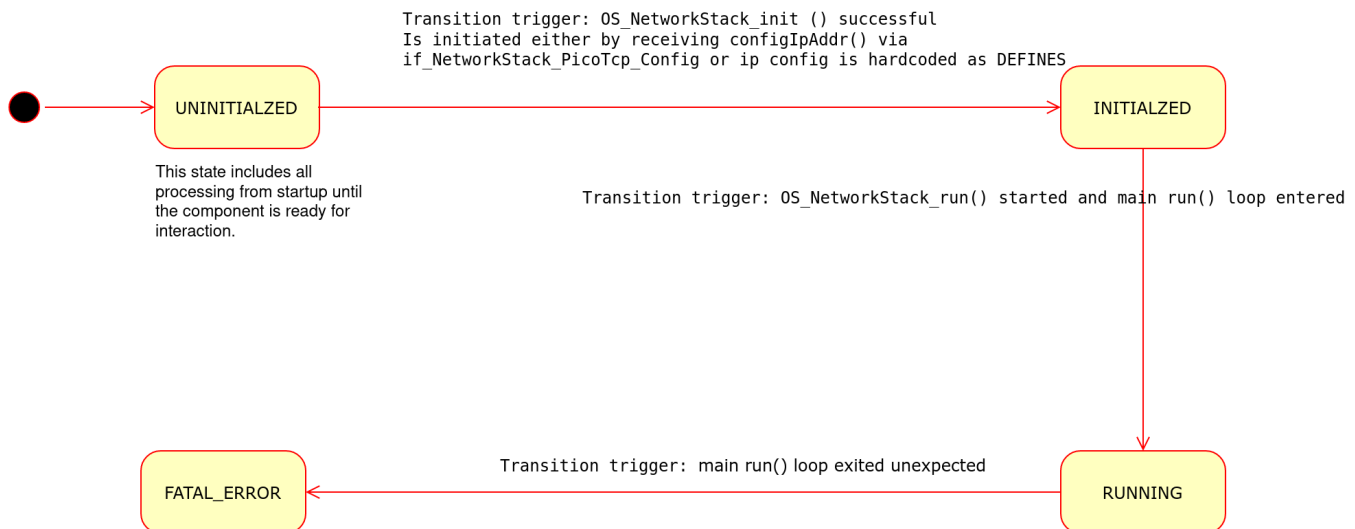
State diagram

Note:

The main issue with the state machine is, that it is valid for a specific implementation - e.g. the NetworkStack_PicoTCP component. But we are implementing the state changes in the `os_network` library.

Another component could implement the `os_network` library differently and could effectively realize a different state machine.

Long term solution can only be, that everything regarding states, transitions, etc. is implemented in the component itself.



Open Points

- new concept for network stack internal signalling / `handle_pico_socket_event()`
 - This callback function is normally called with only one event present.
 - The events should be stored instead of plain overwriting
 - ~~instead sock->event = event_mask;~~
 - ~~do sock->event &= event_mask;~~
 - Remove single events, if they have been processed.
- non-blocking concept (will require RPC call redesign)
- remove RPC functions `socket_bind()` and `socket_listen()`, replace by a new RPC functions that implements `OS_NetworkServerSocket_create()` on network_stack side.
- Should we make use of the ~~CAMEL~~ connector ~~seL4RPCDataportSignal~~? Depending on the notification scheme concept, this connector would offer some benefits but also comes with some constraints:
 - Pro: Since the connector is built on top of the ~~seL4RPCDataport~~ connector, it already offers an autogenerated function for the Server to use to retrieve the Client buffer by its badge ID. This would save us the need to implement it ourselves, similar to what we have done in other components (StorageServer).
 - Pro: By utilizing the associated Notification endpoint, the Server can signal only specific Clients based on their badge ID. This would make it easier on Server side to resolve events happening on specific sockets to the Clients that are linked to these sockets, instead of having to inform all Clients that there are pending events and thereby also waking up Clients that might then just call the RPC to get a status update only to find out that this did not affect them.
 - Cons: This connector does not make use of an actual `seL4Notification` but just uses the RPC endpoint to signal the Clients. Therefore we do not have support for functions like `reg_callback()` that would be very helpful to tie the `get_state_update_rpc()` directly to once there is an incoming event noticed.
 - => not we use **seL4RPCDataport**
- ~~tlsLib~~ and ~~TlsServer~~ modifications
 - from a first quick glance, we might get that to work with minimal changes
 - ~~MBEDTLS~~ does not open the socket or modify its state except read and write
 - ~~MBEDTLS~~ requires us to define callback functions for `recv()` and `send()` via `MBEDTLS_SSL_SET_BIO()`
 - `recv()` may be non-blocking or blocking (actually we are currently using the non-blocking callback, which should be wrong) => so this should enable us to continue using the (adapted) Network API
 - for socket creation, closure, error handling, and notification handling we might need to set up additional infrastructure
 - in ~~OS_Tls_MODE_LIBRARY~~
 - `OS_NetworkSocket_create()` needs to be performed by the user application—so notification handling is the same, as for another component wanting to use the network stack without TLS
 - in ~~OS_Tls_MODE_Client~~

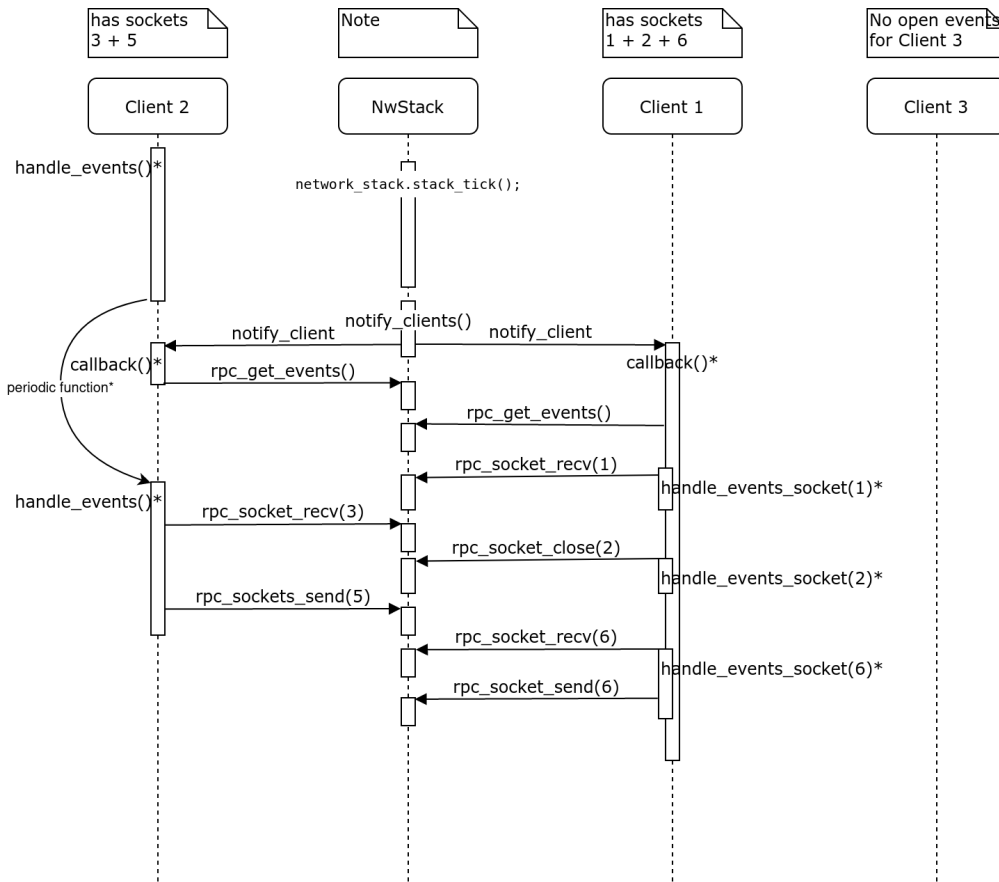
- The TlsServer will require implementation of the new `if_OS_Socket`, like other demos and test systems. Then it will have a non-blocking interface to the network stack.
 - internally the TlsServer uses the `tlsLib`, so again the callback functions for `recv()` and `read()` are required but don't pose an issue
 - The TlsServer provides an RPC interface to its Client. This contains the following functions, which might require modifications:
 - `connect()` => Does a `OS_NetworkSocket_create()` => will block, requires a solution.
 - `disconnect()` => should be fine
 - `handshake()` => might require some time to conclude, so the RPC call blocks for some time, but will only influence the user application, as the TlsServer converts this to non-blocking calls to the network stack
 - `write()` => should be fine
 - `read()` => should be fine
 - `reset()` => should be fine
- Configuration of default network stack component needs to be clarified. `OS_NetworkStack_init()` requires the IP-Address, etc. and this will probably be called in `post_init()` or `run()` of our component.
 - Options are:
 - A) implement optional interface to ConfigServer (probably yes, done it a few times)
 - B) set #defines for that, similar as currently, but rename. Check in the software if set (probably yes)
 - C) implement some RPC interface for that (probably no for first iteration)
- Network Stack naming:
 - "NetworkStackPicoTop" ???
- Branch Struktur während der Umsetzung aufzeigen/ visualisieren

WIP - Notification Scheme Ideas

Idea Notification scheme A

- 1 dataport per connected Client
- 1 notification (=events happened) per connected Client
 - The Notifications will be connected 1:1, so once the NetworkStack will receive an event for one of the active sockets, it will resolve this socket to the specific Client that opened it and only notify this Client about the newly received event.
- 1 additional RPC call `get_pending_events`
 - => will be called by the Client in response to the notification
 - within the RPC the network stack will put a structure into the dataport with the pending events for the sockets of the calling Client
 - struct events
 - `int nr_of_sockets`
 - `Socket_Evt_Status_type socket_events[MAX_NUMBER_SOCKETS]`
 - `Socket_Evt_Status_type`
 - `int socket_handle` => handle of socket, normally known to Client; in case of newly accepted handle, this will be new to the Client, but `parent_socket` should be known
 - `int parent_socket` => handle of parent socket, so Client can address socket correctly
 - `(int state(s))` => states of socket (if required)
 - `bool new_accepted_socket` => Socket was newly accepted by network_stack (only TCP)
 - `bool connect_events_pending` => socket_connect successful (TBC, depends on defined states/rpc calls, if the Client does a `rpc_connect_request()`, either we show here, that this is completed, and a `rpc_connect_result()` needs to be called, or we directly give here the result as an int (0=ok, -1 = general error, -2 = timeout, -3 = connection refused, 1 = waiting, not response yet)
 - `bool receive_events_pending` => data is present, can be read
 - `bool ext_socket_close_request` => network stack or external Client/server want to close the connection (Client needs to call `rpc_socket_close()`)
 - `(bool network_socket_closed)` => impl. socket was closed by network stack (required?)
 - `(bool write_events_pending / write enabled)` => network stack can send data. Not sure if we need this, as this can easily be done by a try to `rpc_socket_send()`.
 - When these events can be reset needs to be defined case by case:
 - `new_accepted` => shall be removed by network stack after `get_pending_events()`
 - `connect` => depends,
 - if a separate `connect_finalize_rpc` exist, than this shall be removed in this rpc;
 - if `get_pending_events` contains all required information, than the network stack shall remove the event after `get_pending_events()`
 - `receive_events_pending` => shall be removed after `rpc_receive / rpc_receive_from` if all presently available data was read
 - `ext_socket_close_request` => shall be in `rpc_socket_close()`
 - `network_socket_closed` => shall be removed by network stack after `get_pending_events()`
 - `write_events_pending / write enabled` => will be removed by `rpc_send / rpc_sendto`
 - As issuing of notifications and requesting of pending events can be timely separated, several bits might be set. In this case a preferred sequence of analysis needs to be defined.
 - The network stack needs to collect events, so no event is being dropped. This should be done in the socket management structure (`OS_NetworkStack_SocketResources_t`)
 - Worst case can be `connect = TRUE` , `receive_events_pending = TRUE` , `ext_socket_close_events = TRUE`
 - In between notification and retrieving pending request, the connection was established, data was received and the ext. the server closed the connection.
 - => read data first

- => call `rpc_socket_close`
- Sending of the events to all connected Clients could be done either:
 - in `OS_NetworkStack_run()` directly following `network_stack.stack_tick();` (preferred)
 - `stack_tick()` processes all open actions within PicoTCP (incl. `handle_pico_socket_event()` callbacks)
 - so after completion, we know that all pending events are known, and we could iterate through all existing sockets and check, which Client needs to be notified (basically check for any open events in `OS_NetworkStack_SocketResources_t`)
 - or directly notify in `handle_pico_socket_event()` callback,
 - this will result in several notifications (not a problem)
 - a problem could be, that some Clients will try to `get_pending_events()` before `network_stack.stack_tick()` has completed
 - maybe this is avoided by the mutex `internal_network_stack_thread_safety_mutex_lock()` but not sure



*client specific implementation

Idea Notification scheme B

The network stack connects to each Client with a message queue and a notification. The message queue allows for async notifications to be delivered from the stack to the Client, without having the Client poll the network stack for the changes.

- 1 dataport
- 1 notification (=events happened) let's call it **EV1**
- 1 message queue (sel4MessageQueue)
 - a second queue will be necessary if we want to allow the user to define custom event filters that don't interfere with normal operations
 - because the existing queue lacks priorities and conditional flushing capabilities
 - feedbackQueue (FQ) and userQueue(UQ)

- **Trentos Socket Events**

```
#define OS_EV_NONE          (0) #define OS_EV_CREATE          (1<=0) #define OS_EV_READ          (1<=1)
#define OS_EV_WRITE        (1<=2) #define OS_EV_CLOSE         (1<=3) #define OS_EV_ERROR        (1<=4)
#define OS_EV_TIMEOUT      (1<=5)
```

Message format for the message queue

```
typedef struct msg{    int? handle; // socket handle, might also be extended to file system    int?
event; // event that happened connected / read / write / closed / error / timeout    int?
accepted_handle; // used to return the Client handle from the accept call    OS_Error_t error_code;
// useful for connect or other async operations or we roll it up in the status} msg_t; // could be
packed to use an int32 for all 4 fields
```

- the network stacks sends msg_t events in the queues
 - the feedback queue (FQ) only gets messages pushed to it as a direct consequence of an RPC operation
 - since it follows a challenge-response pattern, we can guarantee the maximum number of elements in the queue at any given time (number of sockets)
 - async errors that might happen (connection closed) are stored in the network stack until an RPC function is triggered by the client on that socket
 - alternatively we size the FQ in such a way that it can store an error + another event per socket
 - the user queue (UQ) is filled with the messages based on the filters registered by the user
 - if the queue is full, further messages will be dropped and no special handling will be triggered
- triggers the notification
 - we need a message queue and notification because the message queue doesn't implement a callback mechanism
 - the notification triggers its callback which checks the pending events in the queue and triggers the right internal mechanism

TimeServer / Timeouts

- if we connect the Timeserver to the notification event EV1 (by having a TimerReady event callback that emits EV1 events like we did when removing the ticker), we can also handle timeout events.
- If EV1 got triggered but the message queue is empty, it means that a timeout occurred
 - we can't distinguish between multiple events here, and this will need a solution in a multi-threaded component (maybe add a message queue from the timeserver to the component).

New RPC functions

RPC call

```
// to allow the Client app to register an interest in receiving specific messages, we can add an RPC call //
this call will update the userNotificationMask variable for each socket in the network
stackOS_NetworkSocket_setNotificationMask(OS_NetworkSocket_Handle_t handle, int notificationMask); // options
can be for the beginning BLOCKING / NON_BLOCKING // can be extended to set picotcp options ( Nagle for ex.)
OS_NetworkSocket_setSocketOption(OS_NetworkSocket_Handle_t handle, int options, int value);
```

Network Stack

- each **OS_NetworkStack_Socket** socket will include a **userNotificationMask** and an **RPCNotificationMask** variable (and maybe a **defaultMask** if we want non-maskable events)
- on each **handle_pico_socket_event**, the network stack will
 - find the Client to use
 - check if **RPCNotificationMask & defaultMask & pico_event != 0** (if the event that happened is a response to a pending RPC operation)
 - **RPCNotificationMask** is set in the RPC call based on the function that is called and on the socket option
 - for ex. read / recvfrom would set OS_EV_READ (| OS_EV_ERROR) if they're configured as blocking sockets
 - if the result is 0 we don't need to notify anyone and we drop the event here
 - if it's not 0, we create a msg_t struct, fill it with relevant data add it to the message queue and trigger the notification
 - posting to the FQ message queue should never fail
 - for a given socket we might post 1 error in the queue (if we size it accordingly) or keep the error stored in the network stack until the next RPC operation happens on that socket and post it then.
 - check if **userNotificationMask & pico_event != 0** (if the event has a filter installed by the user)
 - we post the corresponding message to the UQ user queue
 - if the queue is full, we drop the message

Network Client

- requires one internal notification event which the EV1 callback will use to notify when the requested event happened (would be 1 / thread in a multi-threaded component)

RPC call

```
// Allows the user to register a callback function for events on a given
socketOS_NetworkSocket_registerCallback(OS_NetworkSocket_Handle_t handle, int event_mask,
function_pointer);
```

- when the EV1 event happens, all messages in the message queues are processed one by one
 - if a callback function is found for them, that function is called
 - if we didn't find a callback, the message is dropped
 - if the message queues are empty it means a timeout occurred (if the TimeServer is connected)
 - blocking functions what are waiting for and _notify call to continue can pass that as a callback function when registering.

RPC Function	Description
socket_create()	• RPC call does all required tasks and doesn't need to create an event in the message queue
socket_close()	• RPC call does all required tasks and doesn't need to create an event in the message queue
socket_connect()	• Client call will block ◦ will register for the event OS_EV_CREATE the unblock function as a callback function • network stack ◦ will set RPCNotificationMask to OS_EV_CREATE ◦ when the event happens, it will add the event to the message queue and trigger the EV1 event
socket_accept()	• Client call will block ◦ will register for the event OS_EV_CREATE the unblock function as a callback function • network stack ◦ will set RPCNotificationMask to OS_EV_CREATE ◦ when the event happens, it will add the event to the message queue and trigger the EV1 event
socket_listen()	• RPC call does all required tasks and doesn't need to create an event in the message queue
socket_bind()	• RPC call does all required tasks and doesn't need to create an event in the message queue
socket_write()	• Client call can block ◦ will register for the event OS_EV_WRITE the unblock function as a callback function • network stack ◦ will set RPCNotificationMask to OS_EV_WRITE ◦ when the event happens, it will add the event to the message queue and trigger the EV1 event
socket_read()	• Client call can block ◦ will register for the event OS_EV_READ the unblock function as a callback function • network stack ◦ will set RPCNotificationMask to OS_EV_READ ◦ when the event happens, it will add the event to the message queue and trigger the EV1 event
socket_sendto()	• Client call can block ◦ will register for the event OS_EV_WRITE the unblock function as a callback function • network stack ◦ will set RPCNotificationMask to OS_EV_WRITE ◦ when the event happens, it will add the event to the message queue and trigger the EV1 event

socket_recvfrom()	• Client call can block ◦ will register for the event OS_EV_READ the unblock function as a callback function • network stack ◦ will set RPCNotificationMask to OS_EV_READ ◦ when the event happens, it will add the event to the message queue and trigger the EV1 event
-------------------	---

Alternative solutions

Custom connector

Design targets / considerations

- avoid copying the data as much as possible
- avoid serializing actions that can be parallelized (copying into/out of the dataport)
- allow multiple threads in a component to use the interface without them unnecessarily blocking one another

Constraints / Limitations

- for each socket there can be at most one pending operation at a given time

Design

RPC

All RPC calls are non-blocking, return as soon as possible, don't transfer any frame data just register operations to be done in the network stack.

Custom connector

We implement a parametrizable custom connector, which has an attribute **number_of_sockets / number_of_connections**. This attribute can be queried by the components it connects to, allowing us to have a single place (1 camkes file) where we define the size.

For each socket, the connector would create 2 dataports (send / receive) and a bi-directional event , allowing the components to signal one another a state change on a given socket.

Client

When creating a socket, one dataports/event pair is reserved for the given socket. Each thread is free to read / write into the socket as it sees hit. Since all RPC calls are non-blocking, blocking behavior will be implemented in the Client side functions by waiting on events on the required socket.

Network stack

The network stack implements a queue implementing the command pattern. This queue has elements added to it by the RPC calls from the various components.

Each element of the queue contains:

- an action type to be performed on a socket (OPEN / CLOSE / READ / WRITE / ...)
- the ID of the component requesting the action
- the socket on which this action is to be performed (this is also used to identify the corresponding buffer pair / event in the custom connector)

Operation

Let's assume we have a component (badge 102) who wants to do a write on its socket 1.

Steps:

- the Client fill the TX dataport of the socket with the data it wants to send
- the Clients calls the socket write RPC for the socket it wants to use
 - if the socket is non-blocking and the data can't be send we return **WOULD_BLOCK**
 - if the socket is non-blocking and the data can be send we call the picotcp send function
 - if the socket is blocking and the data can be send we call the picotcp send function
 - if the socket is blocking and the data can't be send
 - we create a **SEND** entry in the network stacks command queue
 - the Client enters an event wait state
- before each **pico_stack_tick** the network stack checks the command queue for pending operations that can be executed and does so if it finds any. The pending commands it could execute are removed from the queue and the corresponding events are triggered, unblocking the functions blocked on the Client side.

For a read, similar steps would happen, with `handle_pico_socket_event` filling the socket's RX dataport before calling the event to unblock the Client-side.

Simplifying design

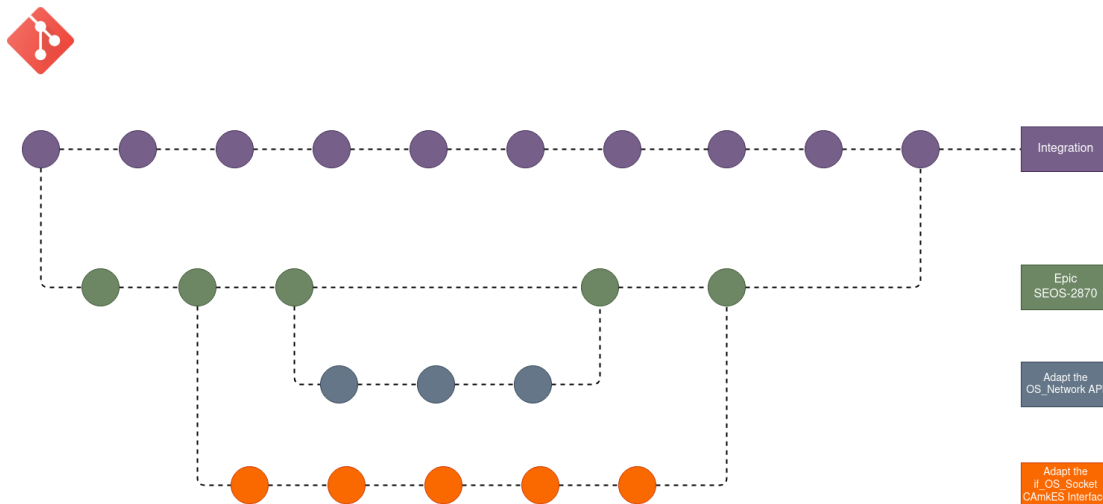
- remove the `ServerSocket` (doesn't really make sense in the context of UDP, and for TCP it just complicates functions)
- remove wrapped functions that hide the BSD standard socket functions (`serversocket_create`)

Development Process

Workflow

The refactoring of the Socket API, CAMkES Interface as well as the creation of the standard `NetworkStack_PicoTcp` SDK component will introduce many changes on many fronts that often cannot be quickly resolved again (meaning test systems / demos are all running and green, definitions of done will not strictly mean "done"). To still be able to come to meaningful DoDs for tickets and have some form of measurable progress, we will need to accept that our development goals will need to be reached in several stages that won't always (especially in the Beginning) result in a shippable state but might still satisfy the agreed DoD.

To match this workflow that will deviate from our standard workflow (in which code change related tickets usually result in a merge to integration), the following flow should be applied:



In all repositories that we need to touch we will create our own intermediate "integration" branch, but it will take the tag of the Epic (e.g. SEOS-2870). All PRs for the tickets we will work on will be filed against this SEOS-2870 Epic branch. This setup should allow us to operate more independently and freely as we are not constrained to our merging rules if we want to merge to integration and should also give us the possibility to create small PRs that are easier to review and integrate into our Epic branch.

Proposed Work Packages

Adapt the API

- Clarify which socket functions should be offered as the API and which functions will be bundled in specific API calls (e.g. `should bind()`, `listen()` be turned into API calls and not bundled)
- Based on the point above, clarify if we still want to keep the `OS_NetworkServerSocket_XXX` scheme or if we can simplify this and not bundle any functions under these API calls.

Replace hardcoded dependency between RPC interface name and library functions

- Our current implementation hides the RPC calls behind the API the Client can make use of, so e.g. `OS_NetworkServerSocket_accept()` will eventually call the `networkStack_rpc_socket_accept()`. This created the problem that the Client has a hard naming dependency on

networkStack_rpc. To improve this and make the design more flexible to various naming options and possibly also a connection to multiple stacks (use case Network Filter that currently had to be implemented with two components connected to two Stack since one cannot connect to two due to the described naming issues), we could make use of a similar pattern that was introduced for components such as the CertServer (IF_CERTSERVER_ASSIGN). This could be done either in a rather simple way by extending and renaming the already existing **OS_NetworkStackClient_SocketDataports_t** that is used to configure the Client library. But this would limit the usage again to only one connected NetworkStack. If we wanted to make this more flexible, more work would need to be put into this adapt the API to be able to make calls to different NetworkStacks (possibly by making use of the currently unused **OS_Network_Context_t** but this needs to be investigated).

- This could be done also early with the current implementation, as the Client-side network API will not dramatically change.

Adapt the if_OS_Socket CAmkES Interface

- The final notification scheme and possible connectors associated with it are not fixed yet, but the current proposals go in the direction of greatly reducing the data ports between the Client (one dataport per Client) and NetworkStack_PicoTcp component and adding a notification between them. Since all RPC functions will be non-blocking in the future, this notification is required to avoid polling on Client side.
- The currently existing network_stack.camkes file in os_network will be removed
 - Macros will be moved either in the NetworkStack_PicoTcp component or into if_OS_socket.camkes, see table below

Macro	File	Comment
IF_OS_SOCKET_PROVIDE	if_OS_Socket.camkes	Declares the interface fields of a component implementing the NwStack side of the SOCKET interface. An open question that still needs to be evaluated is what this macro itself will add to a component using it. One option could be to let this macro itself only bring in the infrastructure required to connect one Client to a server component (1x Notification, 1x Dataport, 1xRPC) and have the default SDK component add the extra 7 sets of dataports and signals to be able to service 8 Clients or if our standard macro here should already be unified for now to bring in 1x RPC and 8x Pairs of Dataport and Notifications so that a component itself would only have to include this macro and not add any additional ports itself. The macro could of course also be extended with some logic that based on some input parameter it will create the necessary infrastructure with some loop macro magic. A component including it would then only have to use it and specify the max amount of Clients.
IF_OS_SOCKET_USE	if_OS_Socket.camkes	Declares the interface fields of a component implementing the user side of the SOCKET interface.
IF_OS_SOCKET_CONNECT_CLIENT	if_OS_Socket.camkes	Connects two components via the SOCKET interface (connectors not final yet but most likely 1x sel4Notification, 1x sel4SharedData, 1x sel4RPCCall (possibly 1x sel4MessageQueue)).
IF_OS_SOCKET_ASSIGN	if_OS_Socket.h	Macro that will take in the rpc and the dataport and is then used on Client side to assign the rpc functions pointers and dataport to a struct that can be given to the API and which the API will use to make the rpc calls and pick the dataport. (Similar as in IF_CERTSERVER_ASSIGN)

Implement handle management based on Client ID

- For the new NwStack, we want a more configurable socket quota on a per-Client basis to reduce the harm that one Client can cause on other Clients connected to the Stack.
- This will require some additional handle management functions especially during the create() and close() phase to assign the handed-out socket handles to a specific Client and make sure it is accounted for against the specific Client socket budget.
- The mapping of sockets to specific Clients will also be required by the whole instance so functions like the main handle_socket_event() loop can derive the Client ID mapped to a specific socket and based on this perform certain checks (like for the accept()) and also possibly signal specific Clients about events on their sockets (in case we use some connector for this case).
- These mapping functionalities could be started early as this could probably already be integrated into the current library state, even if it is not yet then fully used in the handle_pico_socket_event().

Implement the new notification scheme in the os_network_stack library

- The new NetworkStack_PicoTcp component will only offer non-blocking RPC calls and therefore the currently implemented notification scheme that uses several internal notifications on a per-socket basis will no longer be used as the main purpose of these internal notifications was to block the RPC functions until the required event had arrived for the currently active socket within this call.
- The notification scheme concept is not completely fixed yet but it will require a lot fewer internal notifications in the NetworkStack_PicoTcp component, as we do not want to block anywhere in the RPC functions.
- Our callback function handle_pico_socket_event() that we provide to picoTCPs main tick loop to report socket events will need to be modified to no longer issue the emits() using the notifications per socket but instead needs to log the new events received for a socket into a data structure and finally issue one of the notify_events() signals based on which Client is associated with the socket that just received an event tick. The notified Client will then be able to retrieve this previously captured status update through a new RPC get_events() that will be registered to this notification via callback.

Implement the NetworkStack_PicoTcp SDK component

- Add new repo at <https://bitbucket.hensoldt-cyber.systems/projects/SC> called NetworkStack_PicoTcp
- Take network_stack.camkes from the os_network_stack library and **move** it here, rename it to NetworkStack_PicoTcp.camkes.

- Clarify badge numbering scheme (similar to other components starting at 101?).
 - => will probably not possible when using sel4messagequeues, because camkes_msgqueue_channel_register_sender() and camkes_msgqueue_channel_register_receiver() both check for a maximum badge_id / queue_id of 100
 - details will be investigated and solved while implementing Macros: IF_OS_SOCKET_CLIENTS_ASSIGN_BADGES, IF_OS_SOCKET_PROVIDE, IF_OS_SOCKET_USE, IF_OS_SOCKET_CONNECT_CLIENTS
- Finalize the list of macros required for the component. The current expected macros are:

Macro	File	Comment
NetworkStack_COMPONENT_DEFINE	NetworkStack_PicoTcp.camkes	Declares the NwStack component with the following parts: • control; (Active component with a run thread) • consumes EventDataAvailable event_tick_or_data • emits EventDataAvailable event_internal • has mutex allocatorMutex • has mutex nwstackMutex • has mutex socketControlBlockMutex • has mutex stackThreadSafeMutex • uses if_OS_Timer timeServer_rpc • consumes TimerReady timeServer_notify • IF_OS_NIC_USE_WITHOUT_EVENT(nic, nic_port_size) • other_interfaces (Definitions of fields of additional interfaces to which this component could connect to (e.g.: a logger component to log its error messages or ConfigServer). • IF_OS_SOCKET_PROVIDE which includes: ◦ the RPC ◦ 8 Client Signals (e.g. SocketEvent notify_Client1, notify_Client2, notify_Client3 ... notify_Client8) to support 8 Clients ◦ 8 "maybe" dataports (if we follow the approach of 1 dataport per Client and 8 Clients supported for now). • attribute NetworkStack_Config networkStack_config (used to configure the allowed sockets per Client with the NwStack_INSTANCE_CONFIGURE_CLIENTS)
NetworkStack_INSTANCE_CONNECT	NetworkStack_PicoTcp.camkes	Will connect a network stack instance to a ticker event and a NIC component.
NetworkStack_INSTANCE_CONFIGURE	NetworkStack_PicoTcp.camkes	Max number of sockets - is this a define in C-Code we can use to define the size of the array (OS_NetworkStack_CamkesConfig_t internal.number_of_sockets)
NetworkStack_INSTANCE_CONNECT_CLIENTS	NetworkStack_PicoTcp.camkes	One issue we need to solve is, that on the other examples this is being implemented by giving here RPC and dataport of the Clients as attributes. Our current os_network API hides rpc and dataport from the user, e.g. • API: OS_NetworkSocket_write() => get dataport by get_data_port(handle) • calls network_stack_rpc_socket_write(), which is derived from the default rpc name "network_stack_rpc" used on Client and NwStack side. We could use the scheme implemented in if_CertServer.h / IF_CERTSERVER_ASSIGN, resp. CertServer.c / CertServer_addCertToChain().
_SOCKET_BADGE_ASSIGNER	NetworkStack_PicoTcp.camkes	Assign a single badge; this is used internally in the component by the macro NetworkStack_INSTANCE_ASSIGN_CLIENT_BADGES()
NetworkStack_INSTANCE_ASSIGN_CLIENT_BADGES	NetworkStack_PicoTcp.camkes	Assign badges to a list of Clients; badge IDs will start at 101 (TBD) and then be incremented. Clients need to be passed in the same order as they are passed in the NetworkStack_INSTANCE_CONNECT_CLIENTS() macro.

NetworkStack_INSTANCE_CONFIGURE_CLIENTS	NetworkStack_PicoTcp.camkes	Configure the max allowed number of sockets on a per-Client basis. Examples here can be found in the StorageServer component and a possible example for the Client config could be: <pre>struct NetworkStack_ClientConfig { uint8_t socketQuota;}struct NetworkStack_Config { NetworkStack_ClientConfig clients[];}typedef struct{ unsigned int numberOfSockets; OS_Network_Socket_Evt_t socketEvents[MAX_NUMBER_SOCKETS];}OS_Network_User_Evt_t;</pre> It might be necessary to use a new feature CAMKES_CONST_ATTR we introduced to CAMkES here so in code we might need to do something like this to configure the array of sockets on a per client basis: <pre>exampleUser1.socketEvents[CAMKES_CONST_ATTR(clients[1].socketQuota)];</pre>
---	-----------------------------	--

- Clarify how the SDK component can be configured and once fixed, implement it (probably with some optional connection to the ConfigServer or alternatively through some standardized defines similar to what we are currently doing in some demos (demo_network_filter)). In the run() or post_init() function the default component would then have to check if the optional ConfigServer is connected and if yes, retrieve configuration through calls to it (this would also require some standardized naming scheme that we need to document so that a user knows how to prepare a ConfigServer backend with some standardized parameter names to the NetworkStack can retrieve them or else we would need to make it possible to set the customizable parameter names and the domain somewhere). If the ConfigServer is not connected, the NwStack could use some defines we place in the CMakeList file when defining the component (see demo_network_filter).
 - maybe ConfigServer interface: yes
 - maybe LogServer interface: yes
 - => alternatively we could employ a scheme like "other_interfaces" found in network_stack.camkes; but probably difficult as we need to provide the c implementation

Adapt all demo/test systems that use networking to make use of the NwStack SDK component.

- There should not be any real demo/test-specific needs so far regarding the NwStack component (besides maybe the number of required sockets) that would not allow them to use a generic one, so this essentially means switching out the copy-paste component that currently exists across various systems and instead including the generic SDK component.
- Adapt all required settings regarding the Client-specific socket budgets.
- Since our current approach will be to offer only non-blocking functions, for now, the test system and demo implementations will have to be adapted to this non-blocking scheme.
- could be separate tickets (1 for demos, 1 for test systems, 1 for other systems)

Adapt the TLSLib and TLSServer to the new CAMkES Interface of the NwStack

Preliminary expected scope of required changes:

- OS_Tls_MODE_LIBRARY
 - OS_NetworkSocket_create() needs to be performed by the user application - so notification handling is the same, as for another component wanting to use the network stack without TLS
- OS_Tls_MODE_Client
 - The TlsServer will require implementation of the new if_OS_Socket, like other demos and test systems. Then it will have a non-blocking interface to the network stack.
 - internally the TlsServer uses the tlsLib, so again the callback functions for recv() and read() are required but don't pose an issue
 - The TlsServer provides an RPC interface to its Client. This contains the following functions, which might require modifications:
 - connect() => Does a OS_NetworkSocket_create() => will block, requires a solution.
 - disconnect() => should be fine
 - handshake() => might require some time to conclude, so the RPC call blocks for some time, but will only influence the user application, as the TlsServer converts this to non-blocking calls to the network stack
 - write() => should be fine
 - read() => should be fine
 - reset() => should be fine

Update the SDK Handbook documentation, Release Notes, and the migration guide

- Once the code changes are final (or at least in a stable final state where the API, CAMkES component, and macros are not planned to change anymore), we will need to update our handbook documentation.
- Rework and update TRENTO Network API. This will most likely require most of the documentation effort since the page is currently very extensive due to the fact that we did not offer a default component yet and had to be very explicit how our provided library can be wrapped into a component and how it can then be utilized. We might be able to reduce this if we follow a similar approach we have for our other out-of-the-box SDK component and just describe how to pull it into a system, use the macros and configure it.
- Add a section to the Release Notes informing about the newly added SDK component.
- Add a section to the migration guide explaining how users of the previous API and system-specific NwStack component need to adapt to the new API and could use the SDK component. This migration guide will probably also need to cover the changes that had to be done for the TLSServer component / TLSLibrary.

Possible extensions

Extend Client library to support blocking socket calls

- This would then most likely also require additional internal signals on the Client-side to block on.

Extend API with an additional function to set socket options.

- This new function (e.g. **OS_NetworkSocket_setopt()**) should offer the client the possibility to set specific options for a socket he uses (such as disable/enable Nagle algorithm).

Split current network stack library implementation into generic functions and implementation-specific (e.g. picoTCP) parts

- Our current implementation of the network stack library is in part tightly coupled to the picoTCP implementation.
- This coupling should be split so other implementation could easily be integrated with our network stack library.

Add demo "multi-protocol-echo-server"

- Idea about a demo providing 2 TCP server applications and 1 UDP Server, all providing echo capabilities with several Clients
- Could be a WP for Thomas Wittal
- To be merged ?

Proposed new API functions

API Functions

- since any open socket is bound directly to a single network stack instance, it makes sense for the context and handle parameter to be unified in a single parameter
- the distinction between server sockets and normal sockets doesn't make sense, especially since we added UDP support.

Old function	New function	Proposed Change
OS_Error_t OS_NetworkStackClient_init (OS_NetworkStackClientDataports_t* config)	removed	removal

-	OS_Error_t OS_NetworkSocket_getPendingEvents(); • New RPC function required to implement the new notification scheme. • The call will gather all pending events based on the client specific sockets that the NetworkStack has been collecting and place them in a OS_NetworkSocket_Evt_t datastructure in the shared memory between the client and NetworkStack. Possible socket events (might be extended/removed during implementation): <pre>#define OS_SOCK_EV_NONE (0) #define OS_SOCK_EV_CONN_EST (1<<0) // triggered when connection is established (TCP only) following a call to "connect()". #define OS_SOCK_EV_CONN_ACPT (1<<1) // triggered when connection is accepted (TCP only) following a call to "listen()". #define OS_SOCK_EV_READ (1<<2) // triggered when data arrived on the socket #define OS_SOCK_EV_WRITE (1<<3) // triggered when ready to write to the socket (TCP only). Issuing a write/send call will now succeed if the buffer has enough space to allocate new outstanding data. Might not be necessary! #define OS_SOCK_EV_FIN (1<<4) // triggered when the socket is closed (TCP only). No further communication is possible from this point on the socket. #define OS_SOCK_EV_CLOSE (1<<5) // triggered when a FIN segment is received (TCP only). This event indicates that the other endpoint has closed the connection, so the local TCP layer is only allowed to send new data until a local shutdown or close is initiated. #define OS_SOCK_EV_ERROR (1<<6) // triggered when an error occurs</pre> These events would be stored in the event mask of socket event struct: <pre>typedef struct{ int socketHandle; int parentSocket; uint16_t eventMask;} OS_NetworkSocket_Evt_t;</pre> Once the client calls the RPC, all events the NetworkStack has for the specific client will be collected and placed in the dataport. This will be done in such manner that at the beginning of the dataport we will place an integer with the number of events followed by this amount of the struct presented above placed in the port. So in case we would have more events that space in the dataport, we could just place the fitting number, mark these as transfered on NetworkStack side and place the next batch in the next call by the client.
OS_Error_t OS_NetworkSocket_create(OS_NetworkContext_t ctx, OS_NetworkSocket_t* pClientStruct, OS_NetworkSocket_Handle_t* phandle);	OS_Error_t OS_NetworkSocket_create(if_OS_Socket_t ctx, OS_NetworkSocket_t* pClientStruct, OS_NetworkSocket_Handle_t* phandle); • OS_NetworkContext_t is changed to the new if_OS_Socket_t struct that holds all relevant interface ctx information like the vtable for the rpc function calls and the dataport that should be used. • OS_NetworkSocket_Handle_t type is changed to a struct, and is extended with a field representing the network stack instance to be used (if_OS_Socket_t ctx parameter). • no longer masks a call to socket_connect

OS_Error_t OS_NetworkServerSocket_create (OS_NetworkContext_t ctx, OS_NetworkServerSocket_t* pServerStruct, OS_NetworkServer_Handle_t* pSrvHandle)	removed	• OS_NetworkSocket_t extended with fields from OS_NetworkServerSocket_t so it would look something like: <pre>typedef struct{ int domain; int type; uint16_t port; char name[IP_ADD_STR_MAX_LEN + 1]; int backlog;} OS_NetworkSocket_t;</pre> • But this struct only make sense if we really want to pass this around. Many of the API calls could also now be simplified to take in only the parameters they need, e.g. OS_NetworkSocket_create(ctx, handle, domain, type), OS_NetworkSocket_connect(handle, srvAddr,port) • OS_NetworkServerSocket_t removed
OS_Error_t OS_NetworkSocket_close (OS_NetworkSocket_Handle_t handle)	OS_Error_t	no change
OS_Error_t OS_NetworkServerSocket_close (OS_NetworkServer_Handle_t srvHandle)	removed	removal
-	OS_Error_t OS_NetworkSocket_connect (OS_NetworkSocket_Handle_t handle, const void* srvAddr, uint16_t port)	• new function
-	OS_Error_t OS_NetworkSocket_listen (OS_NetworkSocket_Handle_t handle)	• new function • uses the values set when calling create() for backlog

OS_Error_t	OS_Error_t	no change
OS_NetworkSocket_bind (OS_NetworkSocket_Handle_t handle, uint16_t receiving_port)	OS_NetworkSocket_bind (OS_NetworkSocket_Handle_t handle, uint16_t receiving_port)	
OS_Error_t	OS_Error_t	• Rename function and drop Server from naming. • Actually call the accept() underneath and no longer rely on an automatic accep() done by the main loop tick function (currently handle_pico_socket_event()) • OS_NetworkServer_Handle_t will be dropped as described above
OS_NetworkServerSocket_accept (OS_NetworkServer_Handle_t srvHandle, OS_NetworkSocket_Handle_t* phSocket)	OS_NetworkSocket_accept (OS_NetworkSocket_Handle_t srvHandle, OS_NetworkSocket_Handle_t* clientSocket)	
OS_Error_t	OS_Error_t	no change
OS_NetworkSocket_write (OS_NetworkSocket_Handle_t handle, const void* buf, size_t requestedLen, size_t* actualLen)	OS_NetworkSocket_write (OS_NetworkSocket_Handle_t handle, const void* buf, size_t requestedLen, size_t* actualLen)	
noOS_Error_t	OS_Error_t	no change
OS_NetworkSocket_read (OS_NetworkSocket_Handle_t handle, void* buf, size_t requestedLen, size_t* actualLen)	OS_NetworkSocket_read (OS_NetworkSocket_Handle_t handle, void* buf, size_t requestedLen, size_t* actualLen)	

OS_Error_t OS_Netw orkSocket _sendto (OS_Netwo rkSocket_ Handle_t handle, const void* buf, size_t requested Len, size_t * actualLen, OS_N etwork_So cket_t dst_socket)	OS_Error_t OS_Netw orkSocket _sendto (OS_Netwo rkSocket_ Handle_t handle, const void* buf, size_t requested Len, size_t * actualLen, OS_N etwork_So cket_t dst_socket)	Still needs to be clarified if we will keep the OS_Network_Socket_t struct for these UDP functions or if we will change this function similar as the ones above and have it pass the addr and the port as parameters.
OS_Error_t OS_Netw orkSocket _recvfrom (OS_Netwo rkSocket_ Handle_t handle, void* buf, size_t requested Len, size_t * actualLen, OS_N etwork_So cket_t* src_socket)	OS_Error_t OS_Netw orkSocket _recvfrom (OS_Netwo rkSocket_ Handle_t handle, void* buf, size_t requested Len, size_t * actualLen, OS_N etwork_So cket_t* src_socket)	Still needs to be clarified if we will keep the OS_Network_Socket_t struct for these UDP functions or if we will change this function similar as the ones above and have it pass the addr and the port as parameters.

API Use-Cases

TCP Client

Application (user1) connects to Server, sends a request, handles response, and closes the socket

- Get initialized socket handle **OS_NetworkSocket_create(networkStackInterface, &socketStruct, &socketHandle)**
- Request to connect the socket **OS_NetworkSocket_connect(&socketHandle, &socketStruct)**
- Wait on **event_notify_wait()**
- NetworkStack gets an event triggered by the established connection for the socket, stores this event the datastructure it has to collect events for this socket, processes any other events that might have been received for this socket and emits **user1_event_notify_emit()**
- Signal from NetworkStack unblocks the wait, call **OS_NetworkSocket_getPendingEvents()** to gather the datastruct holding all the new events.
 - Check (eventMask & **OS_SOCK_EV_CONN_EST**) to verify that the connection was successfully established.
 - If positive, check (eventMask & **OS_SOCK_EV_WRITE**) and if set, loop over **OS_NetworkSocket_write(&socketHandle...)** until all data is sent or the function returns an error, otherwise wait on **event_notify_wait()** for **OS_SOCK_EV_WRITE**
- Once the request is sent, start reading the response by issuing a call to **OS_NetworkSocket_read(&socketHandle...)** and loop over this function until an error occurs or **actualLenRead == 0**.
 - If more data still needs to be read, wait on **event_notify_wait()** and once (eventMask & **OS_SOCK_EV_READ**), read in more data.
 - Repeat until all required data is read and processed.
- Close the socket with **OS_NetworkSocket_close()**.
 - If we care about the result here, wait on **event_notify_wait()** until we get unblocked and check if (eventMask & **OS_SOCK_EV_CLOSE**) is set to inform us that the socket was closed.

Application connects to Server, tries to send Data but loses connection and tries to reconnect

- Get initialized socket handle **OS_NetworkSocket_create(networkStackInterface, &socketStruct, &socketHandle)**
- Request to connect the socket **OS_NetworkSocket_connect(&socketHandle, &socketStruct)**
- Wait on **event_notify_wait()**
- NetworkStack gets an event triggered by the established connection for the socket, stores this event the datastructure it has to collect events for this socket, processes any other events that might have been received for this socket and emits **user1_event_notify_emit()**

- Signal from NetworkStack unblocks the wait, call **OS_NetworkSocket_getPendingEvents()** to gather the datastruct holding all the new events.
 - Check (eventMask & **OS_SOCK_EV_CONN_EST**) to verify that the connection was successfully established.
 - Check (eventMask & **OS_SOCK_EV_WRITE**) and if set, loop over **OS_NetworkSocket_write(&socketHandle...)** until all data is sent or the function returns an error.
 - Lets assume that in this use case we get to write a few bytes but then the function returns an error that the socket is no longer connected.
 - Reconnect the socket:
 - Close previous socket that lost connection **OS_NetworkSocket_close(&socketHandle)**.
 - Get initialized socket handle **OS_NetworkSocket_create(networkStackInterface, &socketStruct, &newSocketHandle)**
 - For this case it would probably be important to have a quota of at least more than one sockets so that a quick reconnect can be done while the previous socket might still be in a **TIME_WAIT** state.
 - Request to connect the socket **OS_NetworkSocket_connect(&newSocketHandle, &socketStruct)** and wait for the event.
- Wait on **event_notify_wait()**
- NetworkStack gets an event triggered by the established connection for the socket, stores this event the datastructure it has to collect events for this socket, processes any other events that might have been received for this socket and emits **user1_event_notify_emit()**
- Signal from NetworkStack unblocks the wait, call **OS_NetworkSocket_getPendingEvents()** to gather the datastruct holding all the new events.
 - Check (eventMask & **OS_SOCK_EV_CONN_EST**) to verify that the connection was successfully established.
 - Check (eventMask & **OS_SOCK_EV_WRITE**) and if set, loop over **OS_NetworkSocket_write(&socketHandle...)** until all data is sent or the function returns an error.

Application opens several sockets to connect to several Servers, exchanges data.

TCP Server

Server listens for incoming client connections, no client connects

- Get initialized socket handle **OS_NetworkSocket_create(networkStackInterface, &socketStruct, &socketHandle)**
- Bind socket to specific port **OS_NetworkSocket_bind(socketHandle, portNumber)**
- Listen on the bound port **OS_NetworkSocket_listen(socketHandle, backlogSize)**
- Wait on **event_notify_wait()**.

Server listens for incoming client connections, one client connects, sends a short message, and immediately disconnects.

- Get initialized socket handle **OS_NetworkSocket_create(networkStackInterface, socketStruct, socketHandle)**
- Bind socket to specific port **OS_NetworkSocket_bind(socketHandle, portNumber)**
- Listen on the bound port **OS_NetworkSocket_listen(socketHandle, backlogSize)**
- Wait on **event_notify_wait()**.
- NetworkStack gets an event triggered by a new incoming connection. Prepares the **OS_Network_User_Evt_t** datastructure:
 - Place the client handle it in the **OS_Network_User_Evt_t socketEvents[MAX_NUMBER_SOCKETS]**.
 - Set the eventMask | **OS_SOCK_EV_CONN_ACPT** in the new client socket
 - Increment the **numberOfSockets**
 - In this use case we assume that we also quickly receive the read and the close event before the client calls the RPC, so these will also be set for the eventMask for the socket.
 - emit **user1_event_notify_emit()**
- Signal from NetworkStack unblocks the wait, call **OS_NetworkSocket_getPendingEvents()**
- Loop through **OS_Network_User_Evt_t.OS_Network_Socket_Evt_t** with the max **OS_Network_User_Evt_t.numberOfSockets** and for each socket:
 - Check (eventMask & **OS_SOCK_EV_CONN_ACPT**) . This should contain the flag to signal that this client handle has been accepted from the listening queue.
 - Check (eventMask & **OS_SOCK_EV_READ**)
 - Loop over **OS_NetworkSocket_read(&newClientSocketHandle, buffer, sizeof(buffer), &actualLenRead)** while successful and actualLenRead > 0 and process the received data according to the application logic.
 - Check for further events and find that (eventMask & **OS_SOCK_EV_CLOSE**) is set, call **OS_NetworkSocket_close(&newClientSocketHandle)**
- Wait on **event_notify_wait()** for new incoming connections.

Server listens for incoming client connections, several clients connect

- Get initialized socket handle **OS_NetworkSocket_create(networkStackInterface, socketStruct, socketHandle)**
- Bind socket to specific port **OS_NetworkSocket_bind(socketHandle, portNumber)**
- Listen on the bound port **OS_NetworkSocket_listen(socketHandle, backlogSize)**
- Wait on **event_notify_wait()**.
- Signal from NetworkStack unblocks the wait, call **OS_NetworkSocket_getPendingEvents()** to gather the datastruct holding all the new events.
- Loop through **OS_Network_User_Evt_t.OS_Network_Socket_Evt_t** with the max **OS_Network_User_Evt_t.numberOfSockets** which should be at least 1 (could be more in case events arrive for the parent socket) in this case, since the use-case anticipates clients connecting with some time between them and not at once.
 - Check (eventMask & **OS_SOCK_EV_CONN**) . This should contain the **OS_SOCK_EV_CONN_ACPT** flag to signal that this client handle has been accepted from the listening queue.
 - Check (eventMask & **OS_SOCK_EV_READ**)
 - Loop over **OS_NetworkSocket_read(&newClientSocketHandle, buffer, sizeof(buffer), &actualLenRead)** while successful and actualLenRead > 0 and process the received data according to the application logic.
- Wait on **event_notify_wait()**.

- Loop through `OS_Network_User_Evt_t.OS_Network_Socket_Evt_t` with the max `OS_Network_User_Evt_t.numberOfSockets` which should be 1 in this case, since the use-case anticipates clients connecting with some time between them and not at once.
 - Check (eventMask & `OS_SOCK_EV_CONN_ACPT`) . This should contain the flag to signal that this client handle has been accepted from the listening queue.
 - Check (eventMask & `OS_SOCK_EV_READ`)
 - Loop over `OS_NetworkSocket_read(&newClientSocketHandle, buffer, sizeof(buffer), &actualLenRead)` while successful and actualLenRead > 0 and process the received data according to the application logic.
- ...

Server listens for incoming client connections, several clients connect at the same time

- Get initialized socket handle `OS_NetworkSocket_create(networkStackInterface, socketStruct, socketHandle)`
- Bind socket to specific port `OS_NetworkSocket_bind(socketHandle, portNumber)`
- Listen on the bound port `OS_NetworkSocket_listen(socketHandle, backlogSize)`
- Wait on `event_notify_wait()`.
- Signal from NetworkStack unblocks the wait, call `OS_NetworkSocket_getPendingEvents()` to gather the datastruct holding all the new events.
- Loop through `OS_Network_User_Evt_t.OS_Network_Socket_Evt_t` with the max `OS_Network_User_Evt_t.numberOfSockets` and for each socket:
 - Check (eventMask & `OS_SOCK_EV_CONN_ACPT`) . This should contain the flag to signal that this client handle has been accepted from the listening queue.
 - Check (eventMask & `OS_SOCK_EV_READ`)
 - Loop over `OS_NetworkSocket_read(&newClientSocketHandle, buffer, sizeof(buffer), &actualLenRead)` while successful and actualLenRead > 0 and process the received data according to the application logic.

UDP Client

Application (user1) connects to Server, sends a request, handles response, and closes the socket

UDP Server

Server listens for incoming data on a bound port, one client sends some data.

- Get initialized socket handle `OS_NetworkSocket_create(networkStackInterface, socketStruct, socketHandle)`
- Bind socket to specific port `OS_NetworkSocket_bind(socketHandle, portNumber)`
- Check if there is already data to be read `OS_NetworkSocket_rcvfrom(socketHandle...)`
 - If there is data, loop over read until an error occurs or there is no more data to be read.
 - Process the data according to application logic.
- Wait on `event_notify_wait()`
- NetworkStack gets an event triggered by a new read event for the previously bound socket. Prepares the `OS_Network_User_Evt_t` datastructure:
 - Place the socket handle it in the `OS_Network_User_Evt_t socketEvents[MAX_NUMBER_SOCKETS]` .
 - Increments the `numberOfSockets` to 1.
 - Set the eventMask | `OS_SOCK_EV_READ` for this socket.
 - emit `user1_event_notify_emit()`
- Signal from NetworkStack unblocks the wait, call `OS_NetworkSocket_getPendingEvents()` to gather the datastruct holding all the new events.
- Loop through `OS_Network_User_Evt_t.OS_Network_Socket_Evt_t` with the max `OS_Network_User_Evt_t.numberOfSockets` and for each socket:
 - Check (eventMask & `OS_SOCK_EV_READ`)
 - Loop over `OS_NetworkSocket_rcvfrom(socketHandle, ...)` while successful and actualLenRead > 0
 - Process the received data according to the application logic.

Server listens for incoming data on a bound port, several client send data.

- Get initialized socket handle `OS_NetworkSocket_create(networkStackInterface, socketStruct, socketHandle)`
- Bind socket to specific port `OS_NetworkSocket_bind(socketHandle, portNumber)`
- (Optional) Check if there is already data to be read `OS_NetworkSocket_rcvfrom(socketHandle...)`
 - If there is data, loop over read until an error occurs or there is no more data to be read.
 - Process the data according to application logic. Because the UDP socket is **connectionless**, it is able to accept several concurrent connections from different remote hosts. The packet multiplexing must be done in user application.
- Wait on `event_notify_wait()`
- NetworkStack gets an event triggered by a new read event for the previously bound socket. Prepares the `OS_Network_User_Evt_t` datastruct:
 - Place the socket handle it in the `OS_Network_User_Evt_t socketEvents[MAX_NUMBER_SOCKETS]` .
 - Increments the `numberOfSockets`
 - Set the eventMask | `OS_SOCK_EV_READ` for this socket.
 - emit `user1_event_notify_emit()`
- Signal from NetworkStack unblocks the wait, call `OS_NetworkSocket_getPendingEvents()` to gather the datastruct holding all the new events.
- Loop through `OS_Network_User_Evt_t.OS_Network_Socket_Evt_t` with the max `OS_Network_User_Evt_t.numberOfSockets` and for each socket:
 - Check (eventMask & `OS_SOCK_EV_READ`)
 - Loop over `OS_NetworkSocket_rcvfrom(socketHandle, ...)` while successful and actualLenRead > 0
 - Process the data according to application logic. Because the UDP socket is a **connectionless**, it is able to accept several concurrent connections from different remote hosts. The packet multiplexing must be done in user application.

Epic: Non-blocking / single dataport architecture for if_OS_Socket

T Key Status P Summary Resolution Assignee

[Authenticate](#) to retrieve your issues

No issues found

File	Modified
File Idea Notification Scheme A draw.io diagram	May 31, 2021 by thwi104
PNG File Idea Notification Scheme A.png Idea Notification Scheme A exported to image	May 31, 2021 by thwi104
File NwStack Proposed Architecture draw.io diagram	Jun 16, 2021 by Buhl, Matthias
PNG File NwStack Proposed Architecture.png NwStack Proposed Architecture exported to image	Jun 16, 2021 by Buhl, Matthias
File SEOS-2870_workflow draw.io diagram	Jun 17, 2021 by Buhl, Matthias
PNG File SEOS-2870_workflow.png SEOS-2870_workflow exported to image	Jun 17, 2021 by Buhl, Matthias
File NetworkStack_PicoTcp State Machine draw.io diagram	Sep 29, 2021 by thwi104
PNG File NetworkStack_PicoTcp State Machine.png NetworkStack_PicoTcp State Machine exported to image	Sep 29, 2021 by thwi104

Drag and drop to upload or [browse for files](#)
[Download All](#)